



Tags

Django

Python

Desenvolvimento de sistema de registro de ponto em **Python com Django e OpenCV**. Integração de reconhecimento facial em tempo real. Etapas iniciam com registro manual e finalizam com verificação automatizada. Aumento da eficiência e precisão do controle de ponto.

▼ 📖 README.md

👤 Registro de Ponto por Reconhecimento Facial

📝 Descrição do Projeto

Registro de Ponto com Reconhecimento Facial é um sistema desenvolvido em Python com Django e OpenCV para facilitar o registro de presença por meio de reconhecimento facial. Este projeto inclui uma REST API para integração com outros sistemas e um aplicativo desenvolvido em Kivy, permitindo o uso em dispositivos móveis.

⚙️ Funcionalidades

- *Registro de Ponto por Reconhecimento Facial:*
 - 📄 *Funcionários:* Gerenciar informações sobre funcionários, incluindo foto, nome, documento e captura das faces para treinamento.
 - 📷 *Reconhecimento:* Identificação facial do usuário para registrar o ponto automaticamente.
 - 📅 *Histórico de Registros:* Visualização dos registros de ponto por data e hora.
- *Integração com Kivy App:*
 - 📱 *Aplicativo Mobile:* Aplicativo Kivy que se comunica com a API Django para registro de ponto.

- *API REST:*
 -  *Endpoints para Reconhecimento e Cadastro:* Permite o consumo de dados de reconhecimento facial.
 -  *API:* Para operações de registro e consulta de pontos.

Tecnologias Utilizadas

- *Backend:* Django, Django REST Framework, OpenCV
- *App Mobile:* Kivy, kivymd
- *Linguagem:* Python 3.9

Pré-requisitos

Antes de executar o projeto, certifique-se de que os seguintes pré-requisitos estão instalados em sua máquina:

- Python 3.9
- Django (Django==4.2)
- Django REST Framework (djangorestframework==3.15.2)
- Requests (requests==2.32.3)
- OpenCV (opencv-python==4.5.5.64 && opencv-contrib-python==4.5.5.64)
- Numpy (numpy==1.24.4)
- Kivy (para desenvolvimento mobile)

Como Iniciar o Projeto

Siga as instruções para executar o projeto no seu computador (local).

▼ Configuração

▼ Explicação

Versões recentes do Python e OpenCV podem ter incompatibilidades com o reconhecimento facial usando `cv2.face.EigenFaceRecognizer`. Isso ocorre porque:

1. **OpenCV:** A partir das versões mais novas (especialmente da série 4.5.5 em diante), a API `cv2.face` foi gradualmente descontinuada e pode não estar completamente funcional em sistemas modernos.
2. **Python:** O suporte ideal para `cv2.face.EigenFaceRecognizer` tende a funcionar melhor em Python até a versão 3.9. As versões mais recentes, como Python 3.10 ou 3.11, apresentam mudanças que podem causar problemas de compatibilidade ao compilar pacotes como `opencv-contrib-python` e ao utilizar `EigenFaceRecognizer`.

Para projetos que utilizam `cv2.face.EigenFaceRecognizer`, recomenda-se:

- Python **3.9** com OpenCV **4.5.5**.

- Instalar `opencv-contrib-python` correspondente a essa versão para garantir compatibilidade completa.



Para versões mais recentes de Python (3.10+) e OpenCV, onde `cv2.face.EigenFaceRecognizer` pode não funcionar.

Alternativas com OpenCV:

1. **Usar a biblioteca `face_recognition`** para reconhecimento com modelos mais novos.
2. **Integrar OpenCV com `dlib`**: Utilizar `dlib` para detecção e extração de características faciais, combinando essas etapas com OpenCV para manipulação de imagem.

Nesse curso vamos utilizar versão anterior. E entra como feature nova no projeto para atualizar. Vai ser uma demanda interessante.

▼ Configuração no Windows

1. Instalar o Python

<https://www.python.org/downloads/windows/>

<https://www.python.org/downloads/release/python-3913/>

versão 3.9

visual-cpp-build-tools

<https://visualstudio.microsoft.com/pt-br/visual-cpp-build-tools/>

Cmake

<https://cmake.org/download/>

Git Bash

<https://git-scm.com/downloads/win>

Vscode

<https://code.visualstudio.com/>

▼ Configuração no Linux

1. Instalar Dependências do Sistema

Para garantir que todas as dependências estão instaladas:

```
sudo apt update

sudo apt install -y build-essential libssl-dev zlib1g-dev libncurses5-dev \
libncursesw5-dev libreadline-dev libsqlite3-dev libgdbm-dev libdb5.3-dev \
libbz2-dev libexpat1-dev liblzma-dev tk-dev libffi-dev libjpeg-dev \
libfreetype6-dev liblcms2-dev libopenjp2-7-dev libtiff5-dev \
libwebp-dev python3-setuptools python3-pip
```

2. Instalar o Pyenv

Para instalar o Pyenv:

```
curl https://pyenv.run | bash
```

Depois, adicione as linhas abaixo ao seu arquivo `.bashrc` ou `.zshrc`:

```
export PATH="$HOME/.pyenv/bin:$PATH"
eval "$(pyenv init --path)"
eval "$(pyenv init -)"
```

Reinicie o terminal ou recarregue o arquivo:

```
source ~/.bashrc # ou ~/.zshrc
```

3. Instalar a Versão do Python Desejada

Use o pyenv para instalar a versão específica, como Python 3.9:

Exemplo:

```
pyenv install 3.9.20
pyenv global 3.9.20
```

▼ Projeto

▼ 1 - Django / Configurações

Projeto

1. Criar e ativar o ambiente virtual:

```
python -m venv venv
source venv/bin/activate # No Windows: venv\\Scripts\\activate
```

Pyenv (Opcional)

```
pyenv virtualenv .venvGestao
pyenv activate .venvGestao
```

2. Instalar o Django:

```
pip install django
```

3. Criar o projeto Django com o nome "gestao":

```
django-admin startproject gestao .
```

4. Entrar na pasta do projeto:

```
cd gestao
```

5. Iniciar o servidor de desenvolvimento para verificar se está tudo funcionando:

```
python manage.py runserver
```

Acesse `http://127.0.0.1:8000` no navegador para ver o projeto funcionando.

Configurações (settings)

Para configurar o `settings.py`

1. Atualize as variáveis de caminho:

```
import os

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
TEMPLATE_DIR = os.path.join(BASE_DIR, 'templates')
STATIC_DIR = os.path.join(BASE_DIR, 'static')
```

2. Configure o `TEMPLATES` para usar `TEMPLATE_DIR`:

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [TEMPLATE_DIR],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.debug',  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    ],  
]
```

3. Configure o banco de dados como SQLite:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),  
    }  
}
```

4. Defina o idioma e fuso horário:

```
LANGUAGE_CODE = 'pt-br'  
TIME_ZONE = 'America/Sao_Paulo'  
USE_I18N = True  
USE_TZ = True
```

5. Configuração para arquivos estáticos e de mídia:

```
STATIC_URL = '/static/'  
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')  
  
MEDIA_URL = '/media/'  
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

▼ 2 - Inicia Aplicação / Template base

1. Criar a aplicação `registro`:

No terminal, dentro da pasta do projeto, execute:

```
python manage.py startapp registro
```

2. Registrar a aplicação `registro` no `settings.py`:

Abra o arquivo

`settings.py` e adicione `'registro'` em `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    # Apps Django padrão  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    # Aplicação 'registro'  
    'registro',  
]
```

3. Criar uma view de teste no `registro`:

Abra o arquivo

`registro/views.py` e adicione uma função chamada `index`:

```
from django.shortcuts import render  
  
def index(request):  
    return render(request, 'base.html')
```

4. Configurar a URL para a view `index`:

No arquivo

`registro/urls.py`, crie um arquivo se ele não existir, e adicione o seguinte código:

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.index, name='index'),  
]
```

Em seguida, adicione o path da aplicação `registro` no `urls.py` principal do projeto (`gestao/urls.py`):

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),
```

```
path('', include('registro.urls')), # Adicionar
]
```

5. Criar o template `base.html`:

Na cria uma pasta

`templates`, crie o arquivo `base.html` com um código simples de teste:

Bootstrap: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Página de Teste{% endblock %}</title>
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEWih" crossorigin="anonymous">
</head>

<body>
  <div class="container">
    {% block content %}
    <h1>Página de teste!</h1>
    <p>Projeto de Reconhecimento Facial</p>
    {% endblock %}
  </div>

  <!-- Bootstrap, JQuery -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcidslK1eN7N6jIeHz" crossorigin="anonymous"></script>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js" integrity="sha256-/JqT3SQfawRcv/BIHPThkBvs00EvtfFmqPF/1YI/Cxo=" crossorigin="anonymous"></script>

  {% block scripts %}{% endblock scripts %}
</body>
</html>
```

6. Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse `http://127.0.0.1:8000` no navegador.

▼ 3 - Modelo

Passo 1: Criar o modelo `Funcionario`

- `slug`: um campo único, gerado automaticamente no método `save`.
- `foto`: campo de imagem para a foto do funcionário.
- `nome`: campo de texto para o nome do funcionário.
- `cpf`: campo de texto para o CPF.

```
from django.db import models
from django.core.exceptions import ValidationError
from django.utils.text import slugify
from random import randint

class Funcionario(models.Model):
    slug = models.SlugField(max_length=200, unique=True)
    foto = models.ImageField(upload_to='foto/')
    nome = models.CharField(max_length=100)
    cpf = models.CharField(max_length=20)

    def __str__(self):
        return self.nome

    def save(self, *args, **kwargs):
        seq = self.nome + '_FUNC' + str(randint(1000000, 9999999))
        self.slug = slugify(seq)
        super().save(*args, **kwargs)
```

Passo 2: Criar o modelo `ColetaFaces`

- `funcionario`: campo de chave estrangeira para relacionar uma imagem ao funcionário.
- `image`: campo de imagem para salvar a região de interesse (ROI) das faces.

```
class ColetaFaces(models.Model):
    funcionario = models.ForeignKey(Funcionario,
                                    on_delete=models.CASCADE, related_name='funcionario_coletas')
    image = models.ImageField(upload_to='roi/')
```

Passo 3: Criar o modelo `Treinamento`

`modelo`: campo de arquivo para armazenar o classificador de treinamento.

```
class Treinamento(models.Model):
    modelo = models.FileField(upload_to='treinamento/')

    class Meta:
        verbose_name = 'Treinamento'
```

```

        verbose_name_plural = 'Treinamentos'

    def __str__(self):
        return 'Classificadores (frontalface)'

    def clean(self): # Limita a um único arquivo
        model = self.__class__
        if model.objects.exclude(id=self.id).exists():
            raise ValidationError('Só pode haver um arquivo salvo.')

```

Passo 4: Migrar os modelos

```

python manage.py makemigrations
python manage.py migrate

```

Passo 5: Registrar os modelos no Django Admin (opcional)

```

from django.contrib import admin
from .models import Funcionario, ColetaFaces, Treinamento

admin.site.register(Funcionario)
admin.site.register(ColetaFaces)
admin.site.register(Treinamento)

```

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse <http://127.0.0.1:8000/admin/> no navegador.

Uma coisa que gosto de fazer quando tem modelos relacionados é deixar a visualização no django admin dos modelos em inline. (Opcional)

```

from django.contrib import admin
from registro.models import (
    Funcionario, ColetaFaces, Treinamento)

class ColetaFacesInline(admin.StackedInline):
    model = ColetaFaces
    extra = 0

class FuncionarioAdmin(admin.ModelAdmin):
    readonly_fields = ['slug']

```

```

        inlines = (ColetaFacesInline,)

admin.site.register(Funcionario, FuncionarioAdmin)

admin.site.register(Treinamento)

```

▼ 4 - Forms

Passo 1: Criar `forms.py`

Crie ou edite o arquivo `forms.py` para definir os formulários necessários:

1. Formulário para cadastro de funcionário (`FuncionarioForm`)

```

class FuncionarioForm(forms.ModelForm):
    class Meta:
        model = Funcionario
        fields = ['foto', 'nome', 'cpf']

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for field in self.fields.values():
            field.widget.attrs['class'] = 'form-control'

```

2. Formulário para coleta de faces (`ColetaFacesForm`)

```

# DOC: https://docs.djangoproject.com/en/5.1/topics/http/file-uploads/#uploading-multiple-files
# Multiplos Arquivos

class MultipleFileInput(forms.ClearableFileInput):
    allow_multiple_selected = True

class MultipleFileField(forms.FileField):
    def __init__(self, *args, **kwargs):
        kwargs.setdefault("widget", MultipleFileInput())
        super().__init__(*args, **kwargs)

    def clean(self, data, initial=None):
        single_file_clean = super().clean
        if isinstance(data, (list, tuple)):
            result = [single_file_clean(d, initial) for d in data]
        else:
            result = [single_file_clean(data, initial)]
        return result

class ColetaFacesForm(forms.ModelForm):

```

```

images = MultipleFileField()

class Meta:
    model = ColetaFaces
    fields = ['images']

def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    for field in self.fields.values():
        field.widget.attrs['class'] = 'form-control'

```

Passo 2: Criar a view `criar_funcionario`

Crie uma view para cadastrar o funcionário:

Precisamos ter uma instancia para fazer a coleta de faces, por isso que quando finalizamos o envio do formulário é feito o redirect para "form2" `criar_coleta_faces`

```

from django.shortcuts import render, redirect
from .forms import FuncionarioForm, ColetaFacesForm
from .models import Funcionario, ColetaFaces

def criar_funcionario(request):
    if request.method == 'POST':
        form = FuncionarioForm(request.POST, request.FILES)
        if form.is_valid():
            funcionario = form.save()
            return redirect('criar_coleta_faces', funcionario_id=funcionario.id)
    else:
        form = FuncionarioForm()
        return render(request, 'criar_funcionario.html', {'form': form})

```

Passo 3: Criar a view `criar_coleta_faces`

Crie uma view para exibir a coleta de faces após o cadastro do funcionário:

```

def criar_coleta_faces(request, funcionario_id):
    funcionario = Funcionario.objects.get(id=funcionario_id)

    if request.method == 'POST':
        form = ColetaFacesForm(request.POST, request.FILES)
        if form.is_valid():
            # Itera sobre os arquivos enviados e cria uma entrada para cada imagem
            for image in request.FILES.getlist('images'):
                ColetaFaces.objects.create(funcionario=funcionario, image=image)
    else:

```

```

    form = ColetaFACESForm()

    context = {
        'funcionario': funcionario,
        'form': form
    }

    return render(request, 'criar_coleta_faces.html', context)

```

Passo 4: Criar o template `criar_funcionario.html`

Crie um template HTML para o formulário de cadastro do funcionário:

```

{% extends 'base.html' %}
{% block content %}
<div class="row align-items-center justify-content-center">
    <form class="col-md-6 border shadow-sm p-3" method="POST" enctype="multipart/form-data">
        {% csrf_token %}
        <h2>Cadastrar Funcionário</h2>
        {{form.errors}}
        {{form}}
        <button class="btn btn-outline-primary mt-3" type="submit">Salvar e con
tinuar</button>
    </form>
</div>
{% endblock %}

```

Passo 5: Criar o template `criar_coleta_faces.html`

Crie o template para exibir as informações do funcionário e o formulário de coleta de faces:

```

{% extends 'base.html' %}
{% block content %}
<div class="d-flex flex-column flex-lg-row gap-5 justify-content-center p-3">

    <div class="col-md-6 col-lg-4">
        <div class="h4 pb-2 mb-4 text-dark border-bottom border-dark">
            Dados do Funcionário
        </div>
        
        <p><strong> Nome:</strong> {{funcionario.nome}}</p>
        <p><strong> Documento:</strong> {{funcionario.cpf}}</p>
    </div>

    <div class="col-md-6 col-lg-7">
        <form class="mt-3" method="POST" enctype="multipart/form-data"
            action="{% url 'criar_coleta_faces' funcionario.id %}">

```

```

        {{form}}
        <button class="btn btn-lg btn-warning" type="submit">
            Extrair amostras</button>
        </form>
    </div>

</div>
{% endblock %}

```

/registro/urls.py

```

from django.urls import path
from registro.views import criar_funcionario, criar_coleta_faces

urlpatterns = [
    path('', criar_funcionario, name='criar_funcionario'),
    path('criar_coleta_faces/<int:funcionario_id>', criar_coleta_faces, name='criar_coleta_faces')
]

```

```

from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse <http://127.0.0.1:8000/> no navegador.

▼ OpenCV

▼ Conhecendo OpenCV / teste

Documentações:

OpenCV free: <https://opencv.org/university/free-opencv-course/>

OpenCV Tutoriais: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

OpenCV (Open Source Computer Vision Library) é uma biblioteca open-source amplamente usada para tarefas de visão computacional e processamento de imagens. Originalmente desenvolvida pela Intel, ela

é conhecida por sua versatilidade e suporte a várias linguagens, incluindo Python, C++, Java, e integrações com Android, iOS e Web.



Principais Funcionalidades do OpenCV:

1. **Processamento de Imagens:** Inclui operações básicas, como redimensionamento, ajuste de cores, rotação e filtros, além de funções avançadas, como segmentação, morfologia e detecção de bordas.
2. **Análise de Vídeo:** Permite manipulação de vídeos em tempo real, incluindo captura de frames, codecs e operações de pré-processamento. Suporta algoritmos para rastreamento de objetos e contagem de movimentos.
3. **Detecção de Objetos e Rosto:** Ferramentas para identificar rostos, corpos, olhos e outros objetos em imagens. Também permite reconhecimento de rostos utilizando descritores e técnicas de matching de imagens.
4. **Calibração de Câmera e Reconstrução 3D:** Ajusta distorções de câmeras e constrói modelos 3D a partir de imagens 2D, útil em projetos de visão estereoscópica e mapeamento.
5. **Integração com Machine Learning:** Oferece um módulo de machine learning (ML) com classificadores para detecção, segmentação, entre outros. Há suporte nativo para deep learning via frameworks como TensorFlow e PyTorch.

Vamos fazer os testes.

Atualizar `requirements.txt`

```
cmake==3.31.0.1
numpy==1.24.4
opencv-contrib-python==4.5.5.64
pillow==10.4.0
requests==2.32.3
```

1. Crie o diretório para comandos customizados:

No seu app Django, crie o diretório `management/commands` dentro da pasta do app. O caminho seria algo como:

```
registro/
  management/
    commands/
      open_camera.py
```

2. Crie o arquivo `open_camera.py` com o seguinte código:

```

from django.core.management.base import BaseCommand
import cv2

class Command(BaseCommand):
    help = 'Abre a câmera e exibe o vídeo em tempo real'

    def handle(self, *args, **kwargs):
        # Abre a câmera (0 é o índice padrão)
        cap = cv2.VideoCapture(0)

        if not cap.isOpened():
            self.stdout.write(self.style.ERROR('Erro ao abrir a câmera'))
            return

        self.stdout.write(self.style.SUCCESS('Câmera aberta com sucesso. Pressione "q" para sair.'))

        while True:
            # Captura frame por frame
            ret, frame = cap.read()

            if not ret:
                self.stdout.write(self.style.ERROR('Erro ao capturar o frame'))
                break

            # Exibe o frame
            cv2.imshow('Camera', frame)

            # Sai do loop ao pressionar 'q'
            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

        # Libera a câmera e fecha as janelas
        cap.release()
        cv2.destroyAllWindows()
        self.stdout.write(self.style.SUCCESS('Câmera fechada.'))

```

1. Execute o comando:

Para executar o comando, no terminal, rode o seguinte comando dentro do diretório do seu projeto Django:

```
python manage.py open_camera
```

Teste Efetuado, Ok.

▼ 1 - Camera (OpenCV)

Passo 1: Implementar a Classe `VideoCamera`

Primeiro, a classe `VideoCamera` é responsável por capturar o vídeo da câmera. Você já tem uma implementação básica, mas vamos adicionar a detecção de faces com OpenCV. Aqui está a classe ajustada:

`/registro/camera.py`

```
import cv2
import os

class VideoCamera(object):
    def __init__(self):
        self.video = cv2.VideoCapture(0) # Abertura da câmera no meu caso é noteb

        if not self.video.isOpened(): # Verifica se camera está aberta
            print("Erro ao acessar a câmera.")

    def __del__(self):
        self.video.release() # Libera a câmera ao destruir a classe

    def restart(self):
        self.video.release() # Reinicia a câmera
        self.video = cv2.VideoCapture(0) # Cria instancia, se vc não ficar isso a

    def get_camera(self):
        ret, frame = self.video.read() # Leitura do frame
        if not ret: # Verifica se a captura do frame foi bem-sucedida
            print("Falha ao capturar o frame.")
            return None

        # Retorna o frame com a face detectada como imagem JPEG
        ret, jpeg = cv2.imencode('.jpg', frame)
        return jpeg.tobytes() # Converte o frame para formato JPEG em bytes
```

Explicação da Classe `VideoCamera` :

- `__init__` : Inicia a câmera e carrega o classificador Haar Cascade para detecção de faces.
- `get_camera` : Captura um frame da câmera, converte para escala de cinza, detecta as faces e desenha um retângulo ao redor delas.
- `restart` : Libera e reinicia a câmera (caso precise).
- `__del__` : Libera a câmera ao destruir a instância da classe.

Passo 2: Criar o Gerador de Streaming `gen_detect_face`

O gerador `gen_detect_face` será responsável por fornecer os frames com a detecção de faces, que serão transmitidos via HTTP para o navegador.

Vou chamar de `detect_face` por que depois vamos atualizar a camera para detectar um rosto e extrair as faces.

```
from django.http import StreamingHttpResponse
from registro.camera import VideoCamera

camera_detection = VideoCamera() # Instância da classe VideoCamera

# Captura o frame com face detectada
def gen_detect_face(camera_detection):
    while True:
        frame = camera_detection.get_camera()
        if frame is None:
            continue
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')

# Cria streaming para detecção facial
def face_detection(request):
    return StreamingHttpResponse(gen_detect_face(camera_detection),
                                content_type='multipart/x-mixed-replace; boundary=frame')
```

Explicação:

- `gen_detect_face`: Captura os frames com faces detectadas e os envia para o navegador via HTTP em formato JPEG.
- `face_detection`: Configura o streaming com o tipo `multipart/x-mixed-replace`, adequado para enviar imagens contínuas (como um vídeo) para o **navegador**.

`/registro/urls.py`

```
path('face_detection/', face_detection, name='face_detection')
```

Passo 3: Atualizar a View `criar_coleta_faces`

Agora, precisamos passar a URL do `face_detection` para o template na sua view `criar_coleta_faces`. Veja como fazer isso:

```
# Cria coleta de faces (Registro)
def criar_coleta_faces(request, funcionario_id):
    print(funcionario_id) # Identificador do funcionário cadastrado
    funcionario = Funcionario.objects.get(id=funcionario_id) # Resgata o funcionário
```

```

context = {
    'funcionario': funcionario, # Passa o objeto funcionario para o template
    'face_detection': face_detection, # passa camera aqui para renderizar no template
}

return render(request, 'criar_coleta_faces.html', context)

```

Passo 4: Atualizar o Template `criar_coleta_faces.html`

`/registro/template/criar_coleta_faces.html`

```

<div class="col-md-6 col-lg-7">

    <div class="p-3 bg-warning bg-opacity-10 border border-warning rounded mb-3">
        Centralize seu rosto antes de extrair as amostras
    </div>

    

    <form class="mt-3" method="GET" action="{% url 'criar_coleta_faces' funcionario.id %}">

        <button class="btn btn-lg btn-warning" type="submit">
            Extrair amostras</button>

    </form>

</div>

```

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse `http://127.0.0.1:8000/` no navegador.

▼ 2 - Detecção Facial

O classificador `CascadeClassifier` com o arquivo `"haarcascade_frontalface_default.xml"` é usado para detectar faces nos frames capturados pela câmera. Esse arquivo XML contém um modelo treinado especificamente para detectar rostos humanos e é baseado em uma técnica de aprendizado de máquina chamada "Cascade de Haar".

Obter o classificador:

https://github.com/opencv/opencv/blob/4.x/data/haarcascades/haarcascade_frontalface_default.xml



Não esqueça de acessar cursos gratuitos sobre OpenCV. Pra quem está começando é muito bom: <https://opencv.org/>

Passo 1: Atualizar a Classe `VideoCamera`

```
class Camera(object):
    def __init__(self):
        self.face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

        # Retorna o frame modo normal do objeto
    def get_camera(self):
        ret, frame = self.video.read() # Leitura do frame
        if not ret: # Verifica se a captura do frame foi bem-sucedida
            print("Falha ao capturar o frame.")
            return None

        return ret, frame

    # Retorna Camera modo Detecção Facial
    def detect_face(self):
        ret, frame = self.get_camera() # Leitura do frame
        if not ret: # Verifica se a captura do frame foi bem-sucedida
            print("Falha ao capturar o frame.")
            return None

        # Convertendo para escala de cinza para melhorar
        # a detecção de faces
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Detecta faces no frame
        faces = self.face_cascade.detectMultiScale(gray, 1.3, 5)

        # Desenho de um retângulo em torno das faces detectadas
        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2) # R G B

        # Retorna o frame com a face detectada como imagem JPEG
        ret, jpeg = cv2.imencode('.jpg', frame)
        return jpeg.tobytes() # Converte o frame para formato JPEG em bytes
```

Documentação: [OpenCV](https://docs.opencv.org/4.10.0/d4/da8/group_imgcodecs.html)

imencode: https://docs.opencv.org/4.10.0/d4/da8/group_imgcodecs.html

detectMultiScale: https://docs.opencv.org/4.10.0/d1/de5/classcv_1_1CascadeClassifier.html

cvtColor: https://docs.opencv.org/4.10.0/de/d25/imgproc_color_conversions.html

Desenhar: https://docs.opencv.org/4.10.0/dc/da5/tutorial_py_drawing_functions.html

/registro/views.py

```
def gen_detect_face(camera_detection):
    while True: # Codifica o frame como imagem JPEG e envia como resposta HTTP
        frame = camera_detection.detect_face() # Atualiza a aqui
        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')
```

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse <http://127.0.0.1:8000/> no navegador.

Modificações:

Gosto muito de modificar o ROI para criar uma região fixa na camera para o usuário posicionar o rosto.

Quando definimos uma área com `cv2.ellipse`, o usuário tem uma referência visual para alinhar o rosto.

Também com o ROI em escala de cinza e usando a cascata Haar (`self.face_cascade`), a detecção de rostos se torna mais rápida e eficaz.

```
# Retorna Detecção Facial
def detect_face(self):
    ret, frame = self.get_camera() # Leitura do frame
    if not ret: # Verifica se a captura do frame foi bem-sucedida
        print("Falha ao capturar o frame.")
        return None

    # Defina a região de interesse (ROI) onde o rosto será detectado
    altura, largura, _ = frame.shape
    centro_x, centro_y = int(largura/2), int(altura/2)
    a, b = 140, 180
    x1, y1 = centro_x - a, centro_y - b
    x2, y2 = centro_x + a, centro_y + b
    roi = frame[y1:y2, x1:x2]

    # Converta a ROI em escala de cinza para a detecção de faces
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```

# Detecta faces no frame
faces = self.face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

cv2.ellipse(frame, (centro_x, centro_y), (a, b), 0, 0, 360, (0, 0, 255), 10)

for (x, y, w, h) in faces:
    cv2.ellipse(frame, (centro_x, centro_y), (a, b), 0, 0, 360, (0, 255, 0), 10)

# Retorna o frame com a face detectada como imagem JPEG
ret, jpeg = cv2.imencode('.jpg', frame)
return jpeg.tobytes() # Converte o frame para formato JPEG em bytes

```

Documentação: https://docs.opencv.org/4.x/d6/d6e/group_imgproc_draw.html

Pode utilizar a versão quadrado `cv2.rectangle`

```

# Retorna Detecção Facial com Quadrado na ROI
def detect_face(self):
    ret, frame = self.video.read()

    if not ret:
        print("Falha ao capturar o frame.")
        return None

    # Define a região de interesse (ROI) em formato de quadrado
    altura, largura, _ = frame.shape
    centro_x, centro_y = int(largura / 2), int(altura / 2)
    lado = 200 # Tamanho do lado do quadrado (ajuste conforme necessário)

    x1, y1 = centro_x - lado // 2, centro_y - lado // 2
    x2, y2 = centro_x + lado // 2, centro_y + lado // 2
    roi = frame[y1:y2, x1:x2]

    # Converta a ROI em escala de cinza para detecção de faces
    imagemCinza = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)

    # Detecção de rostos apenas na ROI quadrada
    facesDetectadas = self.face_cascade.detectMultiScale(
        imagemCinza, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    # Desenhe o quadrado na imagem original e ao redor do rosto detectado
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 2) # Quadrado vermelho
    for (x, y, w, h) in facesDetectadas:
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2) # Quadrado verde

    # Retorna o frame como imagem JPEG

```

```
ret, jpeg = cv2.imencode('.jpg', frame)
return jpeg.tobytes()
```

▼ 3 - Extração Facial

1. Adicione o evento JavaScript no botão de extração de amostras

Esse evento atualizará o valor do campo `hidden` quando o botão for clicado, para que a view saiba quando o botão foi pressionado.

`registro/templates/criar_coleta_faces.html`

```
<button class="btn btn-lg btn-warning" type="submit"
        onclick="document.getElementById('clicked').value='True';">
    Extrair amostras
</button>
<input type="hidden" id="clicked" name="clicked" value="False">
```

2. Atualize a view `criar_coleta_faces` para tratar a extração

Modifique a view para detectar o clique no botão de extração e chamar a função de extração `face_extract` quando o botão for pressionado.

`registro/views.py`

```
# Função principal da view
def criar_coleta_faces(request, funcionario_id):
    funcionario = Funcionario.objects.get(id=funcionario_id)
    botao_clicado = request.GET.get('clicked', 'False') == 'True'

    context = {
        'funcionario': funcionario,
        'face_detection': face_detection,
        'valor_botao': botao_clicado,
    }

    if botao_clicado:
        print("Cliquei em Extrair Imagens")
        context = face_extract(context, funcionario) # Chama a função de extração

    return render(request, 'criar_coleta_faces.html', context)
```

3. Crie a função de extração de faces

Essa função coleta e salva até 10 amostras faciais de um funcionário na pasta temporária (`tmp`). Verifica também se o limite de coletas foi atingido.

```
def face_extract(context, funcionario):
    num_coletas = ColetaFaces.objects.filter(funcionario__slug=funcionario.slu
```

```

g).count()

    if num_coletas >= 10: # Verifica se limite de coletas foi atingido
        context['erro'] = 'Limite máximo de coletas atingido.'
    else:
        amostra = 0 # Amostras inicial
        numeroAmostras = 10 # Numero de Amostra para extrair
        largura, altura = 220, 220 # largura, altura forma quadradinho
        file_paths = [] # lista de path das amostras

        while amostra < numeroAmostras: # faz um loop até 10 amostra
            ret, frame = camera_detection.get_camera() # pega frame da camera
            (objeto atual)
            crop = camera_detection.sample_faces(frame) # Captura as faces

            if crop is not None: # se não for None,
                amostra += 1 # conta 1

                face = cv2.resize(crop, (largura, altura)) # resize
                imagemCinza = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY) # Passa pa
ra cinza

                # Define o caminho da imagem
                file_name_path = f'./tmp/{funcionario.slug}_{amostra}.jpg' # ex
emplo> leticia_FUNC_XXXX_1.jpg
                cv2.imwrite(file_name_path, imagemCinza)
                file_paths.append(file_name_path) # Adiciona na lista
            else:
                print("Face não encontrada")

            if amostra >= numeroAmostras:
                break # deu 10 amostra para

        camera_detection.restart() # Reinicia a câmera após as capturas

        print(file_paths)

    return context

```

Documentação:

cv2.resize: https://docs.opencv.org/4.10.0/da/d54/group_imgproc_transform.html

cv2.cvtColor: https://docs.opencv.org/4.10.0/db/d64/tutorial_js_colorspaces.html#autotoc_md1615

imencode: https://docs.opencv.org/4.10.0/d4/da8/group_imgcodecs.html

4. Defina a função `sample_faces` na classe `VideoCamera`

Essa função será usada para detectar e retornar o rosto cortado.

No arquivo `camera.py` :

```
class VideoCamera(object):
    def __init__(self):
        self.video = cv2.VideoCapture(0)
        self.face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
        self.img_dir = "./tmp"

        # Garanta que o diretório tmp seja criado
        if not os.path.exists(self.img_dir):
            os.makedirs(self.img_dir) # Cria a pasta `tmp` se não existir

    def sample_faces(self, frame):
        ret, frame = self.get_camera()
        frame = cv2.flip(frame, 180)
        frame = cv2.resize(frame, (480, 360))

        # Detecta a face
        faces = self.face_cascade.detectMultiScale(
            frame, minNeighbors=20, minSize=(30, 30), maxSize=(400, 400))

        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 4)
            cropped_face = frame[y:y+h, x:x+w]
            return cropped_face # Retorna o rosto recortado
```

Documentação:

cv2.flip:

https://docs.opencv.org/4.10.0/d2/de8/group_core_array.html#gaca7be533e3dac7feb70fc60635adf441

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse `http://127.0.0.1:8000/` no navegador.

Mudanças na View (Melhoria)

A função de `face_extract` está ficando muito grande. Vamos melhorar isso assim:

`/registro/views.py`

```
# Cria uma função para extrair e retornar o file_path
def extract(camera_detection, funcionario_slug):
    amostra = 0 # Amostras inicial
    numeroAmostras = 10 # Numero de Amostra para extrair
```

```

    largura, altura = 220, 220 # largura, altura forma quadradinho
    file_paths = [] # lista de path das amostras

    while amostra < numeroAmostras: # faz um loop até 10 amostra
        ret, frame = camera_detection.get_camera() # pega frame da camera (objeto)
        crop = camera_detection.sample_faces(frame) # Captura as faces

        if crop is not None: # se não for None,
            amostra += 1 # conta 1

            face = cv2.resize(crop, (largura, altura)) # resize
            imagemCinza = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY) # Passa para cinza

            # Define o caminho da imagem
            file_name_path = f'./tmp/{funcionario_slug}_{amostra}.jpg' # exemplo>
            cv2.imwrite(file_name_path, imagemCinza)
            file_paths.append(file_name_path) # Adiciona na lista
        else:
            print("Face não encontrada")

    if amostra >= numeroAmostras:
        break # deu 10 amostra para

    camera_detection.restart() # Reinicia a câmera após as capturas
    return file_paths

# Extrai as imagens do funcionário
def face_extract(context, funcionario):
    num_coletas = ColetaFACES.objects.filter(
        funcionario__slug=funcionario.slug).count()

    print(num_coletas) # quantidade de imagens que funcionario tem cadastrado.

    if num_coletas >= 10: # Verifica se limite de coletas foi atingido
        context['erro'] = 'Limite máximo de coletas atingido.'
    else:
        files_paths = extract(camera_detection, funcionario.slug) # passa camera e
        print(files_paths)

    return context

```

5. Atualize a Função `face_extract` para Salvar as Imagens no Banco de Dados

Na função `face_extract`, percorra a lista `files_paths` gerada pela função `extract`, criando instâncias de `ColetaFACES` e salvando cada imagem no modelo.

Em seguida, remova as imagens temporárias da pasta `tmp` e atualize o contexto com as imagens salvas.

```
def face_extract(context, funcionario):
    num_coletas = ColetaFaces.objects.filter(funcionario__slug=funcionario.slug).count()

    if num_coletas >= 10:
        context['erro'] = 'Limite máximo de coletas atingido.'
    else:
        files_paths = extract(camera_detection, funcionario.slug) # Captura faces e retorna paths

        for path in files_paths:
            # Cria uma instância de ColetaFaces e salva a imagem
            coleta_face = ColetaFaces.objects.create(funcionario=funcionario)
            coleta_face.image.save(os.path.basename(path), open(path, 'rb'))
            os.remove(path) # Remove o arquivo temporário após o salvamento

        # Atualiza o contexto com as coletas salvas
        context['file_paths'] = ColetaFaces.objects.filter(funcionario__slug=funcionario.slug)
        context['extracao_ok'] = True # Define sinalizador de sucesso

    return context
```

6. Modifique a View `criar_coleta_faces` para Passar o Contexto para o Template

Certifique-se de que a view chama `face_extract` e envia o contexto atualizado para o template.

```
def criar_coleta_faces(request, funcionario_id):
    funcionario = Funcionario.objects.get(id=funcionario_id)
    botao_clicado = request.GET.get('clicked', 'False') == 'True'

    context = {
        'funcionario': funcionario,
        'face_detection': face_detection,
        'valor_botao': botao_clicado,
    }

    if botao_clicado:
        context = face_extract(context, funcionario) # Chama a função de extração

    return render(request, 'criar_coleta_faces.html', context)
```

7. Atualize o Template para Exibir o Estado da Extração

`registro/template/criar_coleta_faces.html`

```

<div class="col-md-6 col-lg-7">
    {% if not extracao_ok %} <!-- Se nao tiver Extração ok -->
    <!-- CAMERA -->
    <div class="p-3 bg-warning bg-opacity-10 border border-warning rounded
mb-3">
        Centralize seu rosto antes de extrair as amostras
    </div>

    

    <form class="mt-3" method="GET" action="{% url 'criar_coleta_faces' fu
ncionario.id %}">

        <button class="btn btn-lg btn-warning" type="submit"
            onclick="document.getElementById('clicked').value='True';">
            Extrair amostras
        </button>

        <input type="hidden" id="clicked" name="clicked" value="False">

    </form>
    <!-- FINALIZA CAMERA -->

    {% else %} <!-- Se Extração ok -->

    <!-- EXTRAÇÃO OK -->
    <div class="p-3 bg-success bg-opacity-10 border border-success rounded
mb-3">
        Amostras coletadas com sucesso
    </div>

    <h3>Amostras Salvas - Ok</h3>
    <div class="d-flex flex-wrap gap-2">
        {% for file_path in file_paths %}
        <div class="col-2 mb-3">
            <div class="div">
                
                <div class="card-body text-center">
                    <h5 class="card-title">Face {{ forloop.counter }}</h5>
                </div>
            </div>
        </div>
        {% endfor %}
    </div>
    <!-- FINALIZA EXTRAÇÃO -->

```

```

        {% endif %}

        {% if erro %} <!-- Evento no caso do usuario clicar 2 vezes no botão ex
trair. -->
            <h3 class="text-danger">{{ erro }}</h3>
        {% endif %}

    </div>

```

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse <http://127.0.0.1:8000/> no navegador.

▼ 4 - Treinamento Facial

Passo a Passo para Treinamento Facial com OpenCV

Treinamento do Modelo

- `eigenFace.train(np.array(faces), np.array(labels))`: Treina o modelo com as faces e os IDs de funcionário correspondentes.
- `model_filename = "/tmp/classificadorEigen.yml"`: Define o local temporário para salvar o arquivo do modelo.

```

registro/
  management/
    commands/
      treinamento.py

```

`management/commands/treinamento.py`

```

import os
import numpy as np
import cv2
from django.conf import settings
from django.core.files import File
from django.core.management.base import BaseCommand
from registro.models import ColetaFACES, Treinamento

class Command(BaseCommand):
    help = "Treina o classificador Eigen para reconhecimento facial"

```

```

def handle(self, *args, **kwargs):
    self.treinamento_face()

def treinamento_face(self):
    self.stdout.write(self.style.WARNING("Iniciando treinamento com a base de
    print(cv2.__version__)

    # Inicializa o classificador EigenFace
    eigenFace = cv2.face.EigenFaceRecognizer_create(num_components=50, thresh

    faces, labels = [], []
    erro_count = 0

    # Processa cada imagem em ColetaFaces
    for coleta in ColetaFaces.objects.all():
        image_file = coleta.image.url.replace('/media/roi/', '') # leticia-lir
        image_path = os.path.join(settings.MEDIA_ROOT, 'roi', image_file)

        if not os.path.exists(image_path):
            print(f"Caminho não encontrado: {image_path}")
            erro_count += 1
            continue

        # Carrega e processa a imagem
        image = cv2.imread(image_path)
        if image is None:
            print(f"Erro ao carregar a imagem: {image_path}")
            erro_count += 1
            continue

        imagemFace = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        imagemFace = cv2.resize(imagemFace, (220, 220))
        faces.append(imagemFace)
        labels.append(coleta.funcionario.id) # ID funcionario

    # Se não houver faces, interrompe o treinamento
    if not faces:
        print("Nenhuma face encontrada para treinamento.")
        return

    # Realiza o treinamento do modelo
    try:
        eigenFace.train(np.array(faces), np.array(labels))
        print(f"{len(faces)} imagens treinadas com sucesso.")

        # Salva o modelo treinado em um arquivo temporário
        model_filename = "/tmp/classificadorEigen.yml"
        eigenFace.write(model_filename)

```

```

        # Salva o modelo no banco de dados
        with open(model_filename, 'rb') as f:
            treinamento, created = Treinamento.objects.get_or_create()
            treinamento.modelo.save('classificadorEigen.yml', File(f))

        # Remove o arquivo temporário e exibe mensagens de status
        os.remove(model_filename)
        self.stdout.write(self.style.ERROR("Imagens com erro no carregamento"))
        self.stdout.write(self.style.SUCCESS("TREINAMENTO EFETUADO"))

    except Exception as e:
        print(f"Erro durante o treinamento: {e}")

```

A ideia de usar um script separado para o treinamento facial é permitir que o sistema realize essa tarefa automaticamente, em um horário programado, sem depender de um acesso específico à página (view).

Isso traz várias vantagens:

1. **Eficiência:** Ao evitar o treinamento na view, o sistema fica mais leve e rápido para o usuário, pois o treinamento não ocorre cada vez que um novo funcionário é cadastrado.
2. **Rotina Automatizada:** Podemos configurar esse script para rodar em intervalos regulares (por exemplo, diariamente). Dessa forma, qualquer nova imagem ou ajuste no banco de dados será incluído automaticamente no próximo treinamento, garantindo que o modelo de reconhecimento facial esteja sempre atualizado.
3. **Escalabilidade:** Com uma rotina programada, o sistema pode lidar com um número crescente de dados sem interrupções na experiência do usuário.

Teste

management/commands/reconhecimento.py

```

import cv2
import os
from django.core.management.base import BaseCommand
from django.conf import settings
from registro.models import Funcionario, Treinamento

class Command(BaseCommand):
    help = "Comando para teste de reconhecimento facial com exibição ao vivo da c

    def handle(self, *args, **kwargs):
        self.reconhecer_faces()

    def reconhecer_faces(self):
        face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
        reconhecedor = cv2.face.EigenFaceRecognizer_create()

```

```

# Carregar o modelo de treinamento
treinamento = Treinamento.objects.first()
if not treinamento:
    print("Modelo de treinamento não encontrado.")
    return

model_path = os.path.join(settings.MEDIA_ROOT, treinamento.modelo.name)
reconhecedor.read(model_path)

camera = cv2.VideoCapture(0)
largura, altura = 220, 220
font = cv2.FONT_HERSHEY_COMPLEX_SMALL

while True:
    ret, frame = camera.read()
    if not ret:
        print("Erro ao acessar a câmera.")
        break

    frame_redimensionado = cv2.resize(frame, (480, 360))
    imagemCinza = cv2.cvtColor(frame_redimensionado, cv2.COLOR_BGR2GRAY)
    faces_detectadas = face_cascade.detectMultiScale(imagemCinza, minNeigh

    for (x,y,l,a) in faces_detectadas:
        imagemFace = cv2.resize(imagemCinza[y:y+a,x:x+l], (largura, altura))
        cv2.rectangle(frame_redimensionado, (x,y), (x+l,y+a), (0,255,0),2)
        label,result = reconhecedor.predict(imagemFace)
        print(label)
        funcionario = Funcionario.objects.get(id=label)
        if funcionario:
            cv2.putText(frame_redimensionado, str(funcionario.nome).strip(),
        else:
            cv2.putText(frame_redimensionado, "Nenhum user encontrado", (x

    cv2.imshow("Reconhecimento Facial", frame_redimensionado)

    # Parar ao pressionar a tecla 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

camera.release()
cv2.destroyAllWindows()

```

```
python manage.py reconhecimento
```

Na versão mais recente do OpenCV, o módulo `cv2.face` foi completamente removido, e não há substituto direto no próprio OpenCV. Para realizar o reconhecimento facial nas versões mais atuais, você pode usar

uma biblioteca alternativa como `face_recognition` ou `dlib`, que utiliza modelos de redes neurais mais modernos e é fácil de integrar.

▼ Kivy (Aplicativo)

▼ Introdução

Para o reconhecimento facial ao vivo, **a aplicação Django não é a melhor escolha para renderizar diretamente na web**, pois o processo de reconhecimento é muito intensivo em termos de recursos. Renderizar isso em tempo real pelo navegador pode resultar em lentidão.

Solução Alternativa:

Vamos desenvolver uma

aplicação separada para tratar o reconhecimento facial de forma mais eficiente e permitir uma experiência mais fluida para o usuário.

Nos próximos tutoriais, nossa abordagem será:

1. **Desenvolver uma API no Django:** essa API permitirá que dados importantes, como informações do funcionário e o modelo de reconhecimento facial, sejam acessados por outras aplicações. Com isso, conseguimos centralizar a lógica e evitar duplicação de dados.
2. **Criar um aplicativo mobile dedicado:** essa aplicação se conectará à API para capturar e exibir o reconhecimento facial em tempo real, diretamente no dispositivo do usuário. Com uma aplicação dedicada, conseguimos utilizar melhor os recursos do dispositivo, garantindo um reconhecimento mais rápido e eficiente.

App Mobile:

- **Kivy** (Python): facilita a criação de aplicativos com interface gráfica no Android e iOS, com possibilidade de aproveitar o código Python que já estamos usando.
- **ionic** (com Angular, React ou Vue): uma ferramenta de desenvolvimento multiplataforma que permite criar apps nativos e pode consumir dados da nossa API Django.
- **Flutter** (Dart): uma tecnologia popular para apps multiplataforma que permite construir interfaces nativas com alto desempenho em Android e iOS.
- **React Native:** Baseado no **React**, o React Native permite criar aplicativos móveis nativos para iOS e Android utilizando JavaScript (ou TypeScript).

▼ Django Rest Framework

Django REST Framework (DRF) é uma biblioteca poderosa e flexível para criar APIs web em Django. Ela facilita a construção de APIs RESTful com recursos como autenticação, serialização e roteamento.

Documentação: <https://www.django-rest-framework.org/>

Para configurar o Django REST Framework (DRF) em nosso projeto:

Passo 1: Instalar o Django REST Framework

```
pip install djangorestframework
pip install markdown          # Markdown support for the browsable API.
pip install django-filter    # Filtering support
```

Passo 2: Adicionar o DRF no `settings.py`

No arquivo `settings.py`, adicione `'rest_framework'` na lista `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    ...
    'rest_framework', # Adicionando o Django REST Framework
    ...
]
```

Passo 3: Criar (Serializers)

Serializers é a forma como o DRF converte os objetos de banco de dados em formatos como JSON para API.

`api/serializers.py`

```
from rest_framework import serializers
from registro.models import Funcionario, Treinamento

class FuncionarioSerializer(serializers.ModelSerializer):
    class Meta:
        model = Funcionario
        fields = ['id', 'slug', 'foto', 'nome', 'cpf']

class TreinamentoSerializer(serializers.ModelSerializer):
    class Meta:
        model = Treinamento
        fields = ['id', 'modelo']
```

Passo 4: Criar as Views

utilizando o `ModelViewSet` que fornece CRUD completo para os modelos `Funcionario` e `Treinamento`.

`api/views.py`

```
from rest_framework import viewsets
from registro.api.serializers import FuncionarioSerializer, TreinamentoSerializer
from registro.models import Funcionario, Treinamento

class FuncionarioViewSet(viewsets.ModelViewSet):
    queryset = Funcionario.objects.all()
    serializer_class = FuncionarioSerializer
```

```
class TreinamentoViewSet(viewsets.ModelViewSet):
    queryset = Treinamento.objects.all()
    serializer_class = TreinamentoSerializer
```

Passo 5: Configurar as URLs

As URLs do DRF são registradas utilizando o `DefaultRouter`, o que facilita o roteamento das views.

`gestao/urls.py`

```
from django.urls import include, path
from rest_framework.routers import DefaultRouter
from registro.api.views import FuncionarioViewSet, TreinamentoViewSet

router = DefaultRouter()
router.register(r'funcionarios', FuncionarioViewSet)
router.register(r'treinamento', TreinamentoViewSet)

urlpatterns = [
    path('api/', include(router.urls)),
]
```

Verificar se tudo está funcionando:

No terminal, inicie o servidor:

```
python manage.py runserver
```

Agora, acesse `http://127.0.0.1:8000/api/` no navegador.

▼ Kivy O que é / Estrutura de Pastas (Basica)

O **Kivy** é uma biblioteca de código aberto para o desenvolvimento de aplicações multi-touch e interfaces gráficas (GUI) em Python. Ela permite criar aplicativos para **desktop**, **mobile** (iOS, Android) e **embarcados**, com uma única base de código.

Documentação: <https://kivy.org/doc/stable/>

A estrutura de pastas para um projeto **Kivy** pode variar dependendo da complexidade e dos recursos necessários, mas um exemplo básico de organização pode ser o seguinte:

Estrutura Básica do Kivy

```
kivy_project/
|
├── assets/                # Imagens, ícones e arquivos estáticos
|   └── logo.png
```

```

|   └─ background.jpg
|
|   └─ main.py                # Arquivo principal do aplicativo (contém lógica d
e app)
|   └─ main.kv                # Arquivo de layout Kivy, onde a interface é defin
ida
|
|   └─ screens/               # Subpastas para telas, se o app for multi-tela
|       └─ __init__.py
|       └─ screen_one.py      # Exemplo de uma tela
|       └─ screen_two.py      # Outra tela
|
|   └─ widgets/               # Widgets personalizados, se necessário
|       └─ __init__.py
|       └─ custom_button.py   # Exemplo de um widget customizado
|       └─ custom_label.py
|
|   └─ icons/                  # Arquivos de ícones (separados por tipo ou recurs
o)
|       └─ home_icon.png
|       └─ settings_icon.png
|
|   └─ requirements.txt        # Dependências do projeto, incluindo Kivy e KivyMD
|   └─ README.md              # Documentação do projeto

```

Descrição dos diretórios:

1. **assets/** : Contém imagens, ícones e outros arquivos estáticos que o aplicativo pode usar.
2. **main.py** : Contém o código Python principal, onde você define a lógica de comportamento do aplicativo, como carregar telas, iniciar a câmera, realizar ações ao clicar nos botões, etc.
3. **main.kv** : Este arquivo define a interface do usuário usando o **Kivy Language** (KV). É aqui que você descreve a estrutura da UI, como botões, etiquetas, layouts, e outros componentes.
4. **screens/** : Caso o seu aplicativo tenha várias telas, você pode organizar cada tela em um arquivo separado dentro dessa pasta. Cada arquivo de tela conterá a lógica e os widgets necessários para aquela tela específica.
5. **widgets/** : Se você precisar de widgets personalizados ou reutilizáveis em várias partes do aplicativo, pode criar uma pasta **widgets/** para manter esses componentes organizados.
6. **icons/** : Se o aplicativo utilizar muitos ícones, você pode armazená-los em uma pasta dedicada, para organizar melhor os recursos visuais.
7. **requirements.txt** : Aqui você lista as dependências do seu projeto, como o **Kivy**, **KivyMD** e outras bibliotecas necessárias.
8. **README.md** : Arquivo de documentação do projeto onde você pode descrever o propósito do aplicativo, como rodá-lo, e outras instruções relevantes.

Essa estrutura básica pode ser expandida conforme o tamanho e as necessidades do seu projeto, mas com essa organização você consegue manter o código modular e limpo.

vou usar o mesmo repositório do servidor. Vamos fazer algumas modificações nas pastas.

Cria uma pasta `/servidor/` e passa o projeto inteiro django para essa pasta.

Cria uma pasta `/app/` onde será aplicação

▼ Kivy Básico

Primeiros Passos

Criar uma nova virtualenv ou usar a mesma. `pyenv virtualenv .venvAPP && pyenv activate .venvAPP`

Comando de instalação: `pip install kivy`

Documentação: <https://kivymd.readthedocs.io/en/1.0.2/components/>

1. Criação de uma janela básica

`app/kivy/1_app.py`

```
from kivy.app import App
from kivy.uix.label import Label

class MyApp(App):
    def build(self):
        return Label(text='Olá, Letícia!')

if __name__ == '__main__':
    MyApp().run()
```

2. Widgets básicos

`app/kivy/2_widget.py`

```
from kivy.app import App
from kivy.uix.button import Button
from kivy.uix.boxlayout import BoxLayout

class MyApp(App):
    def build(self):
        layout = BoxLayout(orientation='vertical')
        layout.add_widget(Button(text='Botão 1'))
        layout.add_widget(Button(text='Botão 2'))
        return layout

if __name__ == '__main__':
    MyApp().run()
```

Utilizando layout KV.

```
from kivy.app import App
from kivy.lang import Builder

# A string do layout KV
layout = '''
BoxLayout:
    orientation: 'vertical'
    Button:
        text: 'Botão 1'
    Button:
        text: 'Botão 2'
'''

class MyApp(App):
    def build(self):
        return Builder.load_string(layout)

if __name__ == '__main__':
    MyApp().run()
```

3. Eventos e interatividade

- Como usar o `bind()` para manipular eventos.

app/kivy/3_evento.py

```
from kivy.app import App
from kivy.uix.button import Button
from kivy.uix.anchorlayout import AnchorLayout

class MyApp(App):
    def build(self):
        layout = AnchorLayout(anchor_x='center', anchor_y='center')
        button = Button(text='Clique Aqui')
        button.bind(on_press=self.on_button_click)
        layout.add_widget(button)
        return layout

    def on_button_click(self, instance):
        print('Botão clicado!')

if __name__ == '__main__':
    MyApp().run()
```

4. Layouts

- Exemplo de `GridLayout` :

`app/kivy/4_layout.py`

```
from kivy.app import App
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button

class MyApp(App):
    def build(self):
        layout = GridLayout(cols=2)
        layout.add_widget(Label(text='Hello world 1', font_size='20sp'))
        layout.add_widget(Label(text='Hello world 2', font_size='20sp'))
        layout.add_widget(Button(text='Botão 1'))
        layout.add_widget(Button(text='Botão 2'))
        return layout

if __name__ == '__main__':
    MyApp().run()
```

5. Personalização de Widgets

- Uso de propriedades como `size_hint`, `pos_hint`, e `background_color`.

`app/kivy/5_widget_custom.py`

```
from kivy.app import App
from kivy.uix.anchorlayout import AnchorLayout
from kivy.uix.button import Button

class MyApp(App):
    def build(self):
        layout = AnchorLayout(anchor_x='center', anchor_y='center')
        button = Button(text='Clique Aqui', size_hint=(None, None), size=(200,
100), background_color=(1, 0, 0, 1))
        layout.add_widget(button)
        return layout

if __name__ == '__main__':
    MyApp().run()
```

6. Manipulação de Texto

- Como usar widgets de entrada de dados, como `TextInput`.

`app/kivy/6_textinput.py`

```
from kivy.app import App
from kivy.uix.textinput import TextInput
from kivy.uix.button import Button
```

```

from kivy.uix.boxlayout import BoxLayout

class MyApp(App):
    def build(self):
        layout = BoxLayout(orientation='vertical')

        input_field = TextInput(hint_text='Digite algo aqui')
        layout.add_widget(input_field)

        button = Button(text='Imprimir Texto')
        button.bind(on_press=self.on_button_click)
        layout.add_widget(button)

        self.input_field = input_field

        return layout

    def on_button_click(self, instance):
        print(self.input_field.text)

if __name__ == '__main__':
    MyApp().run()

```

7. Imagens

- Como adicionar imagens à interface usando o widget `Image`.

`app/kivy/7_image.py`

```

from kivy.app import App
from kivy.uix.image import Image

class MyApp(App):
    def build(self):
        img = Image(source='./assets/teste.jpg')
        return img

if __name__ == '__main__':
    MyApp().run()

```

Pode fazer uma brincadeira.

```

from kivy.app import App
from kivy.uix.image import Image
from kivy.uix.button import Button
from kivy.uix.boxlayout import BoxLayout

class MyApp(App):
    def build(self):
        layout = BoxLayout(orientation='vertical', padding=20)

```



```

self.img = Image()
layout.add_widget(self.img)

button = Button(text='Mostrar imagem')
button.bind(on_press=self.on_button_click_1)
layout.add_widget(button)

button = Button(text='Remover imagem')
button.bind(on_press=self.on_button_click_2)
layout.add_widget(button)

return layout

def on_button_click_1(self, instance):
    print("Cliquei 1")
    img = './assets/teste.jpg'
    self.img.source = img

def on_button_click_2(self, instance):
    print("Cliquei 2")
    self.img.source = ''

if __name__ == '__main__':
    MyApp().run()

```

8. Animações simples

app/kivy/8_animacao.py

```

from kivy.app import App
from kivy.uix.button import Button
from kivy.uix.anchorlayout import AnchorLayout
from kivy.animation import Animation

class MyApp(App):
    def build(self):
        layout = AnchorLayout(anchor_x='center', anchor_y='center')
        button = Button(text='Clique Aqui', size_hint=(None, None), size=(200,
60), background_color=(1, 0, 0, 1))

        button.bind(on_press=self.on_button_click)
        layout.add_widget(button)

        return layout

    def on_button_click(self, instance):
        print('Botão clicado!')
        self.animate_button(instance)

```

```

def animate_button(self, button):
    anim = Animation(size=(200, 100), duration=1)
    anim.start(button)

if __name__ == '__main__':
    MyApp().run()

```

9. Gerenciamento de Telas (ScreenManager)

app/kivy/9_screen.py

```

from kivy.app import App
from kivy.uix.screenmanager import ScreenManager, Screen
from kivy.uix.button import Button
from kivy.uix.label import Label

class ScreenOne(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.add_widget(Label(text="Tela 1", font_size=40))

        button = Button(text="Ir para Tela 2 >>", size_hint=(None, None), size=(200, 60))
        button.bind(on_press=self.change_screen)
        self.add_widget(button)

    def change_screen(self, instance):
        self.manager.current = 'screen2'

class ScreenTwo(Screen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.add_widget(Label(text="Tela 2", font_size=40))

        button = Button(text="<< Voltar para Tela 1", size_hint=(None, None), size=(200, 60))
        button.bind(on_press=self.change_screen)
        self.add_widget(button)

    def change_screen(self, instance):
        self.manager.current = 'screen1'

class MyApp(App):
    def build(self):
        sm = ScreenManager()
        sm.add_widget(ScreenOne(name='screen1'))
        sm.add_widget(ScreenTwo(name='screen2'))
        return sm

```

```
if __name__ == '__main__':  
    MyApp().run()
```

10. Formulário

app/kivy/10_form.py

```
from kivy.app import App  
from kivy.uix.gridlayout import GridLayout  
from kivy.uix.label import Label  
from kivy.uix.textinput import TextInput  
from kivy.uix.button import Button  
from kivy.uix.checkbox import CheckBox  
from kivy.uix.boxlayout import BoxLayout  
  
class MyForm(GridLayout):  
    def __init__(self, **kwargs):  
        super(MyForm, self).__init__(**kwargs)  
        self.cols = 2  
  
        # Título do Formulário  
self.add_widget(Label(text="Cadastro de Usuário", font_size=24, bold=True,  
self.add_widget(Label()  
  
self.add_widget(Label(text="First Name: "))  
self.name = TextInput(multiline=False)  
self.add_widget(self.name)  
  
self.add_widget(Label(text="Last Name: "))  
self.lastName = TextInput(multiline=False)  
self.add_widget(self.lastName)  
  
self.add_widget(Label(text="Email: "))  
self.email = TextInput(multiline=False)  
self.add_widget(self.email)  
  
# Campo Senha  
self.add_widget(Label(text="Senha:"))  
self.senha_input = TextInput(password=True, hint_text="Digite sua senha")  
self.add_widget(self.senha_input)  
  
# Opção de Aceitação dos Termos  
self.add_widget(Label(text="Aceita os termos de uso?"))  
  
# Layout Horizontal para o CheckBox e o Label  
checkbox_layout = BoxLayout(orientation='horizontal')
```

```

self.checkbox = CheckBox() # Criação do CheckBox
self.checkbox.bind(active=self.on_checkbox_active) # Função que reage ao
checkbox_layout.add_widget(self.checkbox)

self.add_widget(checkbox_layout)

# Botão Enviar
self.submit_button = Button(text="Enviar")
self.submit_button.bind(on_press=self.enviar_formulario)
self.add_widget(self.submit_button)

# Botão Limpar
self.submit_button = Button(text="Limpar")
self.submit_button.bind(on_press=self.limpar_fomrulario)
self.add_widget(self.submit_button)

def enviar_formulario(self, instance):
    nome = self.name.text
    sobrenome = self.lastName.text
    email = self.email.text
    senha = self.senha_input.text
    if self.checkbox.active:
        print(f"Nome: {nome}, Sobrenome: {sobrenome}, Email: {email}, Senha: ")
    else:
        print("Por favor, aceite os termos antes de enviar.")

def limpar_fomrulario(self, instance):
    self.name.text = ''
    self.lastName.text = ''
    self.email.text = ''
    self.senha_input.text = ''
    self.checkbox.active = False
    print("Limpo")

def on_checkbox_active(self, checkbox, value):
    if value:
        print("Checkbox está marcado!")
    else:
        print("Checkbox desmarcado!")

class MyApp(App):
    def build(self):
        return MyForm()

if __name__ == '__main__':
    MyApp().run()

```

KivyMD é uma extensão do Kivy que implementa componentes no estilo Material Design (MD) da Google.

Primeiro Passos

Comando de instalação: `pip install kivymd`

Documentação: <https://kivymd.readthedocs.io/en/latest/components/>

Github Documentação KivyMD: <https://github.com/kivymd/KivyMD/blob/master/examples/appbar.py>

Estrutura do projeto (Layout)

▼ 1 - Configuração

Passo 1 - Vamos passa o arquivo `haarcascade_frontalface_default.xml` **para** `/app/lib/` **Vamos precisar dele para reconhecimento.**

Passo 2 - Cria uma pasta `/assets/` **e na internet abaixe qualquer imagem teste.**

Passo 3 - Cria uma pasta `/tests/` **pois vamos salvar arquivos de teste que vamos fazer.**

Passo 4 - Criar arquivo `requirements.txt` **muito importante.**

Passo 5 - Cria arquivo `app/main.py` **na raiz da pasta**

Passo 6 - Vamos fazer primeiro teste da API

- **Configuração do Ambiente:**

- Certifique-se se projeto Django API esteja rodando no endereço `http://127.0.0.1:8000`.
- Verifique se o módulo `requests` está instalado. Se não estiver, instale-o com:

```
pip install requests
```

- **Criação das Funções:**

- Crie um arquivo Python, `tests/test_api.py`.
- Copie e cole o código das funções `obter_dados_funcionarios` e `obter_classificador` no arquivo.

```
import requests

# API - Lista de Funcionários cadastrado no sistema
def obter_dados_funcionarios():
    response = requests.get('http://127.0.0.1:8000/api/funcionarios/')
    if response.status_code == 200:
        return response.json()
    return []

# Obtem o classificador (Treinamento)
def obter_classificador():
    response = requests.get('http://127.0.0.1:8000/api/treinamento/')
    if response.status_code == 200:
        return response.json()
    return None
```

▼ 2 - Camera (Textura)

1. Inicial

Utilizando tudo que aprendemos, vamos criar uma projeto simples para mostrar a camera. "template" inicial.

`/tests/test_camera.py`

```
from kivymd.app import MDApp
from kivy.core.window import Window
from kivy.uix.image import Image
from kivy.lang import Builder

from kivymd.uix.boxlayout import MDBoxLayout
from kivymd.uix.label import MDLabel
from kivymd.uix.screenmanager import ScreenManager
from kivymd.uix.screen import MDScreen

Window.size = (360, 600)

class MainScreen(MDScreen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs) # Chama o construtor da classe Screen

        layout = MDBoxLayout(orientation="vertical")
        self.add_widget(layout)
```

```

        # Adiciona o widget de imagem
        self.image = Image()
        layout.add_widget(self.image)

    def load_video(self, *args):
        print("Video")

    def open_camera_for_recognition(self):
        # TODO: Remover Widget Label "Clique no botão para reconhecimento"

        # TODO: Abre camera

        print("Abriu a camera")

class ScreenManagerApp(ScreenManager):
    def open_camera_for_recognition(self):
        # Chama o método de MainScreen para abrir a câmera
        self.get_screen('main').open_camera_for_recognition()

class MainApp(MDApp):
    def build(self):
        return Builder.load_string("""
ScreenManagerApp:
    MainScreen:
        name: "main"
        MDLabel:
            text: "Clique no botão para reconhecimento"
            halign: "center"
            theme_text_color: "Secondary"

        MDRaisedButton:
            text: "Iniciar Reconhecimento"
            size_hint: None, None
            size: "200dp", "50dp"
            pos_hint: {"center_x": 0.5, "center_y": 0.1}
            on_press: root.open_camera_for_recognition()

""")

if __name__ == '__main__':
    MainApp().run()

```

2. Função para Ligar a Câmera (`open_camera_for_recognition`)

Ao clicar no botão, iniciar a captura da câmera e programar a exibição em tempo real no aplicativo.

1. Remova mensagens da tela:

- Utilize um loop para remover o widget `MDLabel` de mensagens anteriores.

2. Abra a câmera:

- Inicialize a captura de vídeo com `cv2.VideoCapture(0)`.
- Verifique se a câmera abriu com sucesso.
- Inicie o processo de atualização de vídeo em tempo real com o `Clock.schedule_interval`.

```
from kivy.clock import Clock

import cv2

def open_camera_for_recognition(self):
    # Remove a mensagem (Opcional)
    # camera meio que sobrescreve esse widget.
    for widget in self.children:
        if isinstance(widget, MDLabel):
            self.remove_widget(widget)

    # Iniciar captura de vídeo
    self.cap = cv2.VideoCapture(0) # index 0
    if self.cap.isOpened():
        print("Mostrando a câmera")

        # Chama a função para carregar o vídeo
        Clock.schedule_interval(self.load_video, 1.0 / 60.0)
    else:
        print("Falha ao abrir a câmera")
```

3. Função `load_video` para Atualizar a Imagem

Capturar cada frame da câmera, convertê-lo em textura e exibi-lo no aplicativo.

- Inverta a imagem com `cv2.flip(img, 0)` para alinhar com a exibição em tempo real.
- Converta o frame para o formato de textura.

```
from kivy.graphics.texture import Texture

def load_video(self, *args):
    ret, frame = self.cap.read() # Leitura
    if not ret:
        print("Falha ao capturar o frame")
        return

    # Inverte a imagem e converte para o formato de textura
```



```

        buffer = cv2.flip(frame, 0).tobytes()
        texture = Texture.create(size=(frame.shape[1], frame.shape[0]), colorfmt="bgr")
        texture.blit_buffer(buffer, colorfmt="bgr", bufferfmt="ubyte")
        self.image.texture = texture

```

Verificar se tudo está funcionando:

No terminal, inicie o teste:

```
python test_camera.py
```

▼ 3 - Desenhar Elipse

Aqui, vamos fazer uns testes para colocar um círculo fixo na imagem da camera para que usuário posicione o rosto para reconhecimento. Eu acho que fica muito bonito.

Vamos fazer dois modelos elipse e quadrado.

Instalar numpy: `pip install numpy==1.24.4`

`tests/test_elipse.py`

```

import cv2
import numpy as np

# Defina os parâmetros da imagem
altura, largura = 400, 400
frame = np.zeros((altura, largura, 3), dtype=np.uint8) # Imagem preta

# Defina os parâmetros da elipse
centro_x, centro_y = int(largura / 2), int(altura / 2)
a, b = 140, 180 # Eixos maior e menor
cor = (144, 238, 144) # Verde claro (em BGR)

# Desenhe a elipse com a cor verde claro
cv2.ellipse(frame, (centro_x, centro_y), (a, b), 0, 0, 360, cor, 2)

# Exiba a imagem com a elipse
cv2.imshow("Elipse Verde Claro", frame)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

tests/test_quadrado.py

```
import cv2
import numpy as np

# Defina os parâmetros da imagem
altura, largura = 480, 360
frame = np.zeros((altura, largura, 3), dtype=np.uint8) # Imagem preta

# Defina os parâmetros do quadrado
lado = 140 # Tamanho do lado do quadrado
centro_x, centro_y = int(largura / 2), int(altura / 2)
top_left = (centro_x - lado // 2, centro_y - lado // 2)
bottom_right = (centro_x + lado // 2, centro_y + lado // 2)
cor = (144, 238, 144) # Verde claro (em BGR)

# Desenhe o quadrado
cv2.rectangle(frame, top_left, bottom_right, cor, 2)

# Exiba a imagem com o quadrado
cv2.imshow("Quadrado Verde Claro", frame)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Para testar atualizar função

/tests/test_camera.py

```
def load_video(self, *args):
    ret, frame = self.cap.read() # Leitura
    if not ret:
        print("Falha ao capturar o frame")
        return

    altura, largura, _ = frame.shape

    # Defina os parâmetros da elipse
    centro_x, centro_y = int(largura / 2), int(altura / 2)
    a, b = 140, 180 # Eixos maior e menor
    cor = (144, 238, 144) # Verde claro (em BGR)

    # Desenhe a elipse com a cor verde claro
    cv2.ellipse(frame, (centro_x, centro_y), (a, b), 0, 0, 360, cor, 2)

    # Inverte a imagem e converte para o formato de textura
    buffer = cv2.flip(frame, 0).tobytes()
    texture = Texture.create(size=(frame.shape[1], frame.shape[0]), colorfmt="bgr")
```

```
texture.blit_buffer(buffer, colorfmt="bgr", bufferfmt="ubyte")
self.image.texture = texture
```

Bem legal né?

▼ 4 - Persistir Informações

Para trazer as informações do funcionario depois de reconhecido a ideia de layout foi essa:

tests/test_info.py

```
from kivymd.app import MDApp
from kivy.lang import Builder
from kivymd.uix.screen import MDScreen
from kivy.uix.screenmanager import ScreenManager
from kivy.core.window import Window

from datetime import datetime

Window.size = (340, 680)

class MainScreen(MDScreen):
    def show_recognized_user(self):
        # Define informações fictícias para exibição
        funcionario = {
            "foto": "./assets/teste.jpg",
            "nome": "Letícia Lima",
            "cpf": "123.456.789-00",
        }

        print(funcionario)

class ScreenManagerApp(ScreenManager):
    def show_recognized_user(self):
        # Chama o método de MainScreen para abrir a câmera
        self.get_screen('main').show_recognized_user()

class MainApp(MDApp):
    def build(self):
        return Builder.load_string("""
ScreenManagerApp:
    MainScreen:
        name: "main"

<MainScreen>:
    MDBoxLayout:
        orientation: "vertical"
```

```

        pos_hint: {"center_x": 0.5, "center_y": 0.6}
        MDLabel:
            text: "Clique no botão para reconhecimento"
            halign: "center"
            theme_text_color: "Secondary"

        MDRaisedButton:
            text: "Iniciar Reconhecimento"
            size_hint: None, None
            size: "200dp", "50dp"
            pos_hint: {"center_x": 0.5, "center_y": 0.1}
            on_press: root.show_recognized_user()
    """

if __name__ == '__main__':
    MainApp().run()

```

Depois vamos adicionar as informações no template.

```

# Atribui os valores aos widgets
self.ids.foto.source = funcionario["foto"]
self.ids.nome.text = f"Nome: {funcionario['nome']}"
self.ids.cpf.text = f"CPF: {funcionario['cpf']}"
self.ids.data_hora.text = f>Data e Hora: {datetime.now().strftime('%d/%m/%Y às %H

# Torna o cartão visível
self.ids.card.opacity = 1

```

string

```

MDCard:
    id: card
    size_hint: None, None
    size: "280dp", "300dp"
    pos_hint: {"center_x": 0.5, "center_y": 0.6}
    opacity: 0

    BoxLayout:
        orientation: "vertical"
        padding: "10dp"
        spacing: "10dp"
        AsyncImage:
            id: foto
            size_hint: (1, 0.5)
            pos_hint: {"center_x": 0.5}

```

```

MDLabel:
    id: nome
    adaptive_size: True
    theme_text_color: "Secondary"
    size_hint_y: None
    pos_hint: {"center_x": .5, "center_y": .5}
    padding: "4dp", "4dp"
MDLabel:
    id: cpf
    adaptive_size: True
    theme_text_color: "Secondary"
    size_hint_y: None
    pos_hint: {"center_x": .5, "center_y": .5}
    padding: "4dp", "4dp"
MDLabel:
    id: data_hora
    adaptive_size: True
    theme_text_color: "Secondary"
    size_hint_y: None
    pos_hint: {"center_x": .5, "center_y": .5}
    padding: "4dp", "4dp"

```

Verificar se tudo está funcionando:

No terminal, inicie o teste:

```
python test_info.py
```

Estrutura Projeto Final

▼ Layout do app

Para montar o layout do nosso app vou utilizar todos esses exemplos que fizemos ate agora e vou mexer nas cores etc... mas a estrutura é a mesma.

Pode copiar o codigo base `tests/test_info.py`

Passo 1: Explicação Rápida das Alterações

1. Adicionamos as Classes de Telas:

- Criamos `MainScreen`, `UsuarioScreen` e `ComprovanteScreen`, sendo que `MainScreen` inicia o processo de reconhecimento.

```

class UsuarioScreen(MDScreen):
    pass

```

```
class ComprovanteScreen(MDScreen):
    pass

ScreenManagerApp:
    MainScreen:
    UsuarioScreen:
    ComprovanteScreen:
```

1. Modificamos a Função `show_recognized_user`:

- Quando `show_recognized_user` é chamada, os dados são preenchidos na `UsuarioScreen`, e o app navega automaticamente para a tela "usuario".

`tests/test_screen.py`

```
def show_recognized_user(self):
    # Navegar para a tela 'usuario' usando self.manager
    self.manager.current = 'usuario'

    # Define informações fictícias para exibição
    funcionario = {
        "foto": "https://img.freepik.com/vetores-premium/perfil-de-avatar-ilus",
        "nome": "Letícia Lima",
        "cpf": "123.456.789-00",
    }

    print(funcionario)

    # Atribui os valores aos widgets na tela UsuarioScreen
    usuario_screen = self.manager.get_screen('usuario')
    usuario_screen.ids.foto.source = funcionario["foto"]
    usuario_screen.ids.nome.text = f"Nome: {funcionario['nome']}"
    usuario_screen.ids.cpf.text = f"CPF: {funcionario['cpf']}"
    usuario_screen.ids.data_hora.text = f>Data e Hora: {datetime.now().strftime(

        # Torna o cartão visível na tela 'usuario'
    usuario_screen.ids.card.opacity = 1
```

KV de MenuScreen

```
<MainScreen>:
    name: 'main'

MDScreen:
    md_bg_color: 0.941, 0.957, 0.973, 1

MDTopAppBar:
    title: "Reconhecimento"
```

```

        specific_text_color: 1, 1, 1, 1
        anchor_title: "center"
        md_bg_color: 0.173, 0.243, 0.314, 1
        elevation: 0.5
        pos_hint: {"top": 1}

MDBoxLayout:
    orientation: "vertical"
    adaptive_size: True
    spacing: "20dp"
    pos_hint: {"center_x": 0.5, "center_y": 0.6}

MDCard:
    id: headimage
    size_hint: None, None
    size: "300dp", "300dp"
    pos_hint: {"center_x": 0.5}

    AsyncImage:
        size_hint: (1, 1)
        pos_hint: {'center_x': 0.5}
        source: './assets/teste.jpg'

MDRaisedButton:
    text: 'Registrar'
    font_size: '20sp'
    pos_hint: {'center_x': 0.5, 'center_y': 0.25}
    md_bg_color: 1, 0.388, 0.278, 1
    size_hint: (0.7, 0.1)
    elevation: 0.5
    on_press: root.show_recognized_user()

```

`ComprovanteScreen` contém um botão para voltar à `MainScreen`, e a `UsuarioScreen` contém um botão para ir à tela de comprovante.

KV de UsuarioScreen

```

<UsuarioScreen>:
    name: 'usuario'
    MDScreen:
        md_bg_color: 0.941, 0.957, 0.973, 1
        MDTopAppBar:
            title: "Usuário Identificado"
            specific_text_color: 1, 1, 1, 1
            anchor_title: "center"
            md_bg_color: 0.173, 0.243, 0.314, 1
            elevation: 0.5

```

```

        pos_hint: {"top": 1}
MDCard:
    id: card
    size_hint: None, None
    size: "280dp", "300dp"
    pos_hint: {"center_x": 0.5, "center_y": 0.6}
    opacity: 0
    BoxLayout:
        orientation: "vertical"
        padding: "10dp"
        spacing: "10dp"
        AsyncImage:
            id: foto
            size_hint: (1, 0.5)
            pos_hint: {"center_x": 0.5}
        MDLabel:
            id: nome
            adaptive_size: True
            theme_text_color: "Secondary"
            size_hint_y: None
            pos_hint: {"center_x": .5, "center_y": .5}
            padding: "4dp", "4dp"
        MDLabel:
            id: cpf
            adaptive_size: True
            theme_text_color: "Secondary"
            size_hint_y: None
            pos_hint: {"center_x": .5, "center_y": .5}
            padding: "4dp", "4dp"
        MDLabel:
            id: data_hora
            adaptive_size: True
            theme_text_color: "Secondary"
            size_hint_y: None
            pos_hint: {"center_x": .5, "center_y": .5}
            padding: "4dp", "4dp"
MDRaisedButton:
    text: 'Confirmar'
    font_size: '20sp'
    pos_hint: {'center_x': 0.5, 'center_y': 0.25}
    size_hint: (0.7, 0.1)
    elevation: 0.5
    md_bg_color: 0.298, 0.686, 0.314, 1
    on_press: root.manager.current = 'comprovante'
MDRaisedButton:
    text: 'Não sou eu'
    font_size: '20sp'
    pos_hint: {'center_x': 0.5, 'center_y': 0.1}
    size_hint: (0.7, 0.1)

```



```
elevation: 0.5
md_bg_color: 0.9, 0.3, 0.3, 1
on_press: root.manager.current = 'main'
```

KV de ComprovanteScreen

```
<ComprovanteScreen>:
  name: 'comprovante'
  MDScreen:
    md_bg_color: 0.941, 0.957, 0.973, 1
    MDTopAppBar:
      title: "Comprovante"
      specific_text_color: 1, 1, 1, 1
      anchor_title: "center"
      md_bg_color: 0.173, 0.243, 0.314, 1
      elevation: 0.5
      pos_hint: {"top": 1}
    MDCard:
      id: card_comprovante
      size_hint: None, None
      size: "280dp", "300dp"
      md_bg_color: 1.0, 0.976, 0.912, 1
      pos_hint: {"center_x": 0.5, "center_y": 0.6}
      opacity: 1
      BoxLayout:
        orientation: "vertical"
        padding: "10dp"
        spacing: "10dp"
        MDLabel:
          text: 'Comprovante'
          halign: 'center'

    MDRaisedButton:
      text: 'Fechar'
      font_size: '20sp'
      pos_hint: {'center_x':0.5, 'center_y':0.2}
      md_bg_color: 1, 0.388, 0.278, 1
      size_hint: (0.7, 0.1)
      elevation: 0.5
      on_press: root.manager.current = 'main'
```

Documentação:

MDLabel: <https://kivymd.readthedocs.io/en/1.1.1/components/label/>

MDScreen: <https://kivymd.readthedocs.io/en/1.1.1/components/screen/>

BoxLayout: <https://kivymd.readthedocs.io/en/1.1.1/components/boxlayout/>

Card: <https://kivymd.readthedocs.io/en/1.1.1/components/card/>

Button: <https://kivymd.readthedocs.io/en/1.1.1/components/button/>

Layout Final que vamos utilizar.

```
from kivymd.app import MDApp
from kivy.lang import Builder
from kivymd.uix.screen import MDScreen
from kivymd.uix.screenmanager import ScreenManager
from kivy.core.window import Window

from datetime import datetime

Window.size = (340, 680)

class MainScreen(MDScreen):
    def show_recognized_user(self):
        # Navegar para a tela 'usuario' usando self.manager
        self.manager.current = 'usuario'

        # Define informações fictícias para exibição
        funcionario = {
            "foto": "https://img.freepik.com/vetores-premium/perfil-de-avatar-ilus",
            "nome": "Letícia Lima",
            "cpf": "123.456.789-00",
        }

        print(funcionario)

        # Atribui os valores aos widgets na tela UsuarioScreen
        usuario_screen = self.manager.get_screen('usuario')
        usuario_screen.ids.foto.source = funcionario["foto"]
        usuario_screen.ids.nome.text = f"Nome: {funcionario['nome']}"
        usuario_screen.ids.cpf.text = f"CPF: {funcionario['cpf']}"
        usuario_screen.ids.data_hora.text = f>Data e Hora: {datetime.now().strftime('%d/%m/%Y %H:%M')}

        # Torna o cartão visível na tela 'usuario'
        usuario_screen.ids.card.opacity = 1

class UsuarioScreen(MDScreen):
    pass

class ComprovanteScreen(MDScreen):
    pass

class ScreenManagerApp(ScreenManager):
    def show_recognized_user(self):
        # Chama o método de MainScreen para abrir a câmera
```

```

        self.get_screen('main').show_recognized_user()

class MainApp(MDApp):
    def build(self):
        return Builder.load_string("""
ScreenManagerApp:
    MainScreen:
    UsuarioScreen:
    ComprovanteScreen:

<MainScreen>:
    name: 'main'

    MDScreen:
        md_bg_color: 0.941, 0.957, 0.973, 1

        MDTopAppBar:
            title: "Reconhecimento"
            specific_text_color: 1, 1, 1, 1
            anchor_title: "center"
            md_bg_color: 0.173, 0.243, 0.314, 1
            elevation: 0.5
            pos_hint: {"top": 1}

            MDBoxLayout:
                orientation: "vertical"
                adaptive_size: True
                spacing: "20dp"
                pos_hint: {"center_x": 0.5, "center_y": 0.6}

                MDCard:
                    id: headimage
                    size_hint: None, None
                    size: "300dp", "300dp"
                    pos_hint: {"center_x": 0.5}

                    AsyncImage:
                        size_hint: (1, 1)
                        pos_hint: {'center_x': 0.5}
                        source: './assets/teste.jpg'

                MDRaisedButton:
                    text: 'Registrar'
                    font_size: '20sp'
                    pos_hint: {'center_x': 0.5, 'center_y': 0.25}
                    md_bg_color: 1, 0.388, 0.278, 1
                    size_hint: (0.7, 0.1)

```

```

        elevation: 0.5
        on_press: root.show_recognized_user()

<UsuarioScreen>:
    name: 'usuario'
    MDScreen:
        md_bg_color: 0.941, 0.957, 0.973, 1
        MDTopAppBar:
            title: "Usuário Identificado"
            specific_text_color: 1, 1, 1, 1
            anchor_title: "center"
            md_bg_color: 0.173, 0.243, 0.314, 1
            elevation: 0.5
            pos_hint: {"top": 1}
        MDCard:
            id: card
            size_hint: None, None
            size: "280dp", "300dp"
            pos_hint: {"center_x": 0.5, "center_y": 0.6}
            opacity: 0
            BoxLayout:
                orientation: "vertical"
                padding: "10dp"
                spacing: "10dp"
                AsyncImage:
                    id: foto
                    size_hint: (1, 0.5)
                    pos_hint: {"center_x": 0.5}
                MDLabel:
                    id: nome
                    adaptive_size: True
                    theme_text_color: "Secondary"
                    size_hint_y: None
                    pos_hint: {"center_x": .5, "center_y": .5}
                    padding: "4dp", "4dp"
                MDLabel:
                    id: cpf
                    adaptive_size: True
                    theme_text_color: "Secondary"
                    size_hint_y: None
                    pos_hint: {"center_x": .5, "center_y": .5}
                    padding: "4dp", "4dp"
                MDLabel:
                    id: data_hora
                    adaptive_size: True
                    theme_text_color: "Secondary"
                    size_hint_y: None
                    pos_hint: {"center_x": .5, "center_y": .5}
                    padding: "4dp", "4dp"

```

```

MDRaisedButton:
  text: 'Confirmar'
  font_size: '20sp'
  pos_hint: {'center_x': 0.5, 'center_y': 0.25}
  size_hint: (0.7, 0.1)
  elevation: 0.5
  md_bg_color: 0.298, 0.686, 0.314, 1
  on_press: root.manager.current = 'comprovante'
MDRaisedButton:
  text: 'Não sou eu'
  font_size: '20sp'
  pos_hint: {'center_x': 0.5, 'center_y': 0.1}
  size_hint: (0.7, 0.1)
  elevation: 0.5
  md_bg_color: 0.9, 0.3, 0.3, 1
  on_press: root.manager.current = 'main'

<ComprovanteScreen>:
  name: 'comprovante'
  MDScreen:
    md_bg_color: 0.941, 0.957, 0.973, 1
    MDTopAppBar:
      title: "Comprovante"
      specific_text_color: 1, 1, 1, 1
      anchor_title: "center"
      md_bg_color: 0.173, 0.243, 0.314, 1
      elevation: 0.5
      pos_hint: {"top": 1}
    MDCard:
      id: card_comprovante
      size_hint: None, None
      size: "280dp", "300dp"
      md_bg_color: 1.0, 0.976, 0.912, 1
      pos_hint: {"center_x": 0.5, "center_y": 0.6}
      opacity: 1
      BoxLayout:
        orientation: "vertical"
        padding: "10dp"
        spacing: "10dp"
        MDLabel:
          text: 'Comprovante'
          halign: 'center'

    MDRaisedButton:
      text: 'Fechar'
      font_size: '20sp'
      pos_hint: {'center_x': 0.5, 'center_y': 0.2}
      md_bg_color: 1, 0.388, 0.278, 1
      size_hint: (0.7, 0.1)

```

```

        elevation: 0.5
        on_press: root.manager.current = 'main'
    """

if __name__ == '__main__':
    MainApp().run()

```

▼ Reconhecimento

▼ Passo 1: Implementando a Câmera no Projeto

Vamos agora implementar a câmera no nosso projeto, dentro do layout final que desenvolvemos.

Primeiro, vamos iniciar a câmera. Vocês já aprenderam como fazer isso, certo?

Crie um arquivo `main.py` na raiz da pasta, que é nosso aplicativo oficial. Copie o código do layout final que acabamos de criar.

Adicionaremos uma função `__init__` para configurar a câmera:

```

# Novos imports
import cv2

from kivy.uix.image import Image
from kivy.graphics.texture import Texture
from kivy.clock import Clock

from kivymd.uix.boxlayout import MDBoxLayout
from kivymd.uix.label import MDLabel

class MainScreen(MDScreen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs) # Chama o construtor da classe Screen

        layout = MDBoxLayout(orientation="vertical", pos_hint={"center_x": 0.
5, "center_y": 0.6})
        self.add_widget(layout) # Adiciona o layout à tela

        # Adiciona o widget de imagem
        self.image = Image()
        layout.add_widget(self.image)

        # TODO: Baixar o modelo de treinamento e carregar no reconhecedor

```

Vamos agora criar a função para carregar o vídeo da câmera:

```

def load_video(self, *args):
    ret, frame = self.cap.read()

```

```

if not ret:
    print("Falha ao capturar o frame")
    return

# Defina a região de interesse (ROI) onde o rosto será detectado
altura, largura, _ = frame.shape
centro_x, centro_y = int(largura / 2), int(altura / 2)
a, b = 140, 180 # Ajuste o tamanho da elipse conforme necessário
x1, y1 = centro_x - a, centro_y - b
x2, y2 = centro_x + a, centro_y + b

# Desenha a elipse na ROI
cv2.ellipse(frame, (centro_x, centro_y), (a, b), 0, 0, 360, (144, 238, 144), 6)

# Exibe a imagem capturada com a elipse
buffer = cv2.flip(frame, 0).tobytes()
texture = Texture.create(size=(frame.shape[1], frame.shape[0]), colorfmt="bgr")
texture.blit_buffer(buffer, colorfmt="bgr", bufferfmt="ubyte")
self.image.texture = texture

# TODO: Adicionar lógica de reconhecimento facial dentro da ROI

```

Para permitir que o usuário abra a câmera e inicie o reconhecimento facial, vamos criar uma função de clique:

```

def open_camera_for_recognition(self):
    # Oculta a imagem principal
    self.ids.headimage.opacity = 0

    # Remove a mensagem (opcional)
    for widget in self.children:
        if isinstance(widget, MDLabel):
            self.remove_widget(widget)

    # Inicia a captura de vídeo
    self.cap = cv2.VideoCapture(0) # index 0 para a câmera padrão
    if self.cap.isOpened():
        print("Câmera aberta")

        # Agenda a função para carregar o vídeo
        Clock.schedule_interval(self.load_video, 1.0 / 60.0)

        # TODO: Agendar o início do reconhecimento facial após 5 segundos

    else:
        print("Falha ao abrir a câmera")

```

Não esqueça de alterar a chamada agora é para reconhecimento primeiro `open_camera_for_recognition`

```
class ScreenManagerApp(ScreenManager):
    def open_camera_for_recognition(self):
        # Chama o método de mainScreen para abrir a câmera
        self.get_screen('main').open_camera_for_recognition()
```

Evento de click também.

<MainScreen>

```
MDRaisedButton:
    text: 'Registrar'
    font_size: '20sp'
    pos_hint: {'center_x': 0.5, 'center_y': 0.25}
    md_bg_color: 1, 0.388, 0.278, 1
    size_hint: (0.7, 0.1)
    elevation: 0.5
    on_press: root.open_camera_for_recognition()
```

Verificar se tudo está funcionando:

No terminal, inicie o teste:

```
python main.py
```

▼ Passo2: Implementando a Lógica de Reconhecimento Facial

Agora, vamos implementar a lógica de reconhecimento facial dentro do arquivo `__init__`. Para isso, precisaremos de um classificador. Usaremos a API para buscar o primeiro objeto de treinamento.

Primeiro, importe o `tempfile` para criar um arquivo temporário para o classificador.

```
# Novos Imports
import tempfile
import requests

class MainScreen(MDScreen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        ...

    # Carrega o classificador frontal do OpenCV
    self.face_cascade = cv2.CascadeClassifier("../lib/haarcascade_frontalf
ace_default.xml")
    self.reconhecedor = cv2.face.EigenFaceRecognizer_create()
```



```

        # Baixa o modelo de treinamento e carrega no reconhecedor
        treinamento = requests.get("http://127.0.0.1:8000/api/treinamento/").
json()
        model_url = treinamento[0]['modelo'] # Pega o primeiro item da lista

        with tempfile.NamedTemporaryFile(delete=True) as temp_file:
            temp_file.write(requests.get(model_url).content)
            temp_file.flush()
            self.reconhecedor.read(temp_file.name) # Carrega o modelo no rec
onhecedor

```

Windows:

```

import os

tmp_dir = "./tmp"
os.makedirs(tmp_dir, exist_ok=True)

...
# Caminho do arquivo temporário no diretório local
tmp_path = os.path.join(tmp_dir, "modelo.xml")

# Salva o modelo no diretório tmp
with open(tmp_path, "wb") as temp_file:
    temp_file.write(requests.get(model_url).content)
    self.reconhecedor.read(temp_file.name) # Carrega o modelo no reconhecedor

```

Agora, vamos atualizar a função `load_video` para incluir a lógica de reconhecimento facial.

```

def load_video(self, *args):
    ...

    roi = frame[y1:y2, x1:x2] # Extraí a região de interesse

    # Adicionar lógica de reconhecimento facial dentro da ROI
    imagemCinza = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    facesDetectadas = self.face_cascade.detectMultiScale(imagemCinza, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    # Para cada rosto detectado, realiza o reconhecimento facial
    for (x, y, w, h) in facesDetectadas:
        imagemFace = cv2.resize(imagemCinza[y:y+h, x:x+w], (220, 220))
        label, confianca = self.reconhecedor.predict(imagemFace)

        print(f"ID reconhecido: {label}")

    # Exibe o resultado do reconhecimento no frame

```

```

        if label:
            print("Reconhecimento")

        # TODO: Adicionar lógica para obter dados do funcionário aqui

        break # Interrompe o loop após o primeiro rosto reconhecido, se
necessário

```

Verificar se tudo está funcionando:

No terminal, inicie o teste antes ative o servidor por q agora tamos utilizando a API também.

```
python main.py
```

Legal, está puxando o identificador do funcionario certo ? Agora vamos fazer uma busca na API para encontrar esse funcionario.

MainScreen declarar variaveis que vamos utilizar.

```

class MainScreen(MDScreen):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.recognized_user = None # Retorna usuario
        self.recognized = False # status do reconhecimento
        self.recognition_enabled = False # Variável para controlar o início do

```

Na função `load_video` atualizamos:

```

def load_video(self, *args):
    ...

    # Obtém dados do funcionário
    response = requests.get(f"http://127.0.0.1:8000/api/funcionarios/{label}/")
    if response.status_code == 200: # Retorno é OK,
        funcionario = response.json()
        print(funcionario)

        self.recognized_user = funcionario['id']
        self.recognized = True

        # TODO: Enviar dados para UsuarioScreen após reconhecimento

        Clock.unschedule(self.load_video) # Interrompe o loop após identificaçã

        # TODO: Liberar a camera depois de utilizar para primeiro reconheciment

```

```
break # Interrompe o loop após o primeiro rosto reconhecido, se necessário
```

▼ Passo 3: Reiniciar a Câmera Após o Reconhecimento

Após o reconhecimento do primeiro usuário, a câmera está travando porque o loop é interrompido com `break`, mas não há um reset para liberar a câmera e permitir o reconhecimento de uma nova pessoa.

Vamos implementar uma função para liberar a câmera após o primeiro reconhecimento:

```
def reset_camera(self):
    # Restaura a opacidade da imagem inicial e libera a câmera
    self.ids.headimage.opacity = 1 # Restaura a opacidade da imagem inicial

    # Libera o recurso da câmera e remove a textura
    if self.cap:
        self.cap.release() # Libera a câmera
    self.image.texture = None
```

Atualizando a Função `load_video`

Agora, vamos chamar `reset_camera` ao final da lógica de reconhecimento na função `load_video`:

```
# Liberar a camera depois de utilizar para primeiro reconhecimento
self.reset_camera()
```

▼ Passo 4: Enviar os dados do usuário para tela de confirmação

Agora precisamos passar `"funcionario"` para função `show_recognized_user` precisamos fazer essa atualização aqui.

```
def show_recognized_user(self, funcionario):
    # Navegar para a tela 'usuario' usando self.manager
    self.manager.current = 'usuario'

    # Atribui os valores aos widgets na tela UsuarioScreen
    usuario_screen = self.manager.get_screen('usuario')
    usuario_screen.ids.foto.source = funcionario["foto"]
    usuario_screen.ids.nome.text = f"Nome: {funcionario['nome']}"
    usuario_screen.ids.cpf.text = f"CPF: {funcionario['cpf']}"
    usuario_screen.ids.data_hora.text = f>Data e Hora: {datetime.now().strftime}

    # Torna o cartão visível na tela 'usuario'
    usuario_screen.ids.card.opacity = 1
```

Atualizar a função `load_video`

Agora `load_video` está completa e atende o que precisamos por enquanto.

```
# Enviar dados para UsuarioScreen após reconhecimento
self.show_recognized_user(funcionario)
```

Quando usuário faz uma ação de Confirmar ou Não sou eu precisa voltar para main com a camera funcionando para proximo reconhecimento, certo ?

Então precisamos fazer umas mudanças no evento do botão.

<UsuarioScreen>

Botão Não sou eu.

```
MDRaisedButton:
    text: 'Não sou eu'
    font_size: '20sp'
    pos_hint: {'center_x': 0.5, 'center_y': 0.1}
    size_hint: (0.7, 0.1)
    elevation: 0.5
    md_bg_color: 0.9, 0.3, 0.3, 1
    on_press:
        root.manager.current = 'main'
        root.manager.get_screen('main').reset_camera()
```

<ComprovanteScreen>

Botão Fechar

```
MDRaisedButton:
    text: 'Fechar'
    font_size: '20sp'
    pos_hint: {'center_x': 0.5, 'center_y': 0.2}
    md_bg_color: 1, 0.388, 0.278, 1
    size_hint: (0.7, 0.1)
    elevation: 0.5
    on_press:
        root.manager.current = 'main'
        root.manager.get_screen('main').reset_camera()
```

Tempo de Reconhecimento

Perceberam que está muito rapido o reconhecimento, não da nem tempo de olhar pra camera rsrs.

Vamos da uma controlada nisso só para testar mesmo, cria uma função `start_recognition`

```
def start_recognition(self, *args):
    # Ativa o reconhecimento após o tempo de espera
```

```
self.recognition_enabled = True
```

Agora, vamos chamar `start_recognition` ao final da lógica de reconhecimento na função `open_camera_for_recognition`:

```
# TODO: Agendar o início do reconhecimento facial após 5 segundos
Clock.schedule_once(self.start_recognition, 5)
```

Vamos colocar um time de 5 segundos é tempo suficiente para usuario colocar o rosto e reconhecer.

```
def load_video(self, *args):
    ...
    # Clock 5 segundos
    if not self.recognition_enabled:
        return # False
```

▼ Passo 5: Salvar dados do usuário Reconhecido no Banco de Dados

1. Criação do Modelo no Servidor

Crie o modelo `RegistroFuncionario` que relaciona o funcionário com a data de registro:

```
from django.db import models

class RegistroFuncionario(models.Model):
    funcionario = models.ForeignKey(Funcionario, on_delete=models.CASCADE)
    data_hora = models.DateTimeField()

    def __str__(self):
        return f"{self.funcionario.nome} - {self.data_hora}"
```

admin.py

```
admin.site.register(RegistroFuncionario)
```

2. Criação da API

Adicione uma view e serializer para salvar os registros:

Serializer:

api/serializers.py

```
from rest_framework import serializers
from .models import RegistroFuncionario

class RegistroFuncionarioSerializer(serializers.ModelSerializer):
    class Meta:
```

```
model = RegistroFuncionario
fields = ['funcionario', 'data_hora']
```

View:

api/views.py

```
from .models import RegistroFuncionario
from .serializers import RegistroFuncionarioSerializer

class RegistroFuncionarioViewSet(viewsets.ModelViewSet):
    queryset = RegistroFuncionario.objects.all()
    serializer_class = RegistroFuncionarioSerializer
```

URL:

gestao/urls.py

```
from .views import RegistroFuncionarioAPIView

router.register(r'registros', RegistroFuncionarioViewSet)
```

Rodar: `python manage.py runserver`

Aplicativo

Função em `UsuarioScreen`

Na tela `UsuarioScreen`, crie a função para enviar os dados:

```
from kivy.clock import mainthread
import requests
from datetime import datetime

class UsuarioScreen(MDScreen):
    def confirmar_registro(self):
        if not self.funcionario_id:
            print("Funcionário não definido.")
            return

        funcionario = {
            "funcionario": self.funcionario_id, # Envia o ID para API
            "data_hora": self.data_hora # Data
        }

        url_api = "http://127.0.0.1:8000/api/registros/"

        try:
```

```

        response = requests.post(url_api, json=funcionario)

        if response.status_code == 201:
            print("Registro salvo com sucesso!")
            # Navegar para a tela de comprovante
            self.manager.current = 'comprovante'

        else:
            print("Erro ao salvar registro:", response.json())

    except Exception as e:
        print("Erro na conexão com a API:", e)

```

1. Atualizar o Componente **MDRaisedButton**

Atualize o botão na tela **UsuarioScreen** para chamar a função **confirmar_registro**:

```

<UsuarioScreen>:
    name: "usuario"
    MDRaisedButton:
        text: 'Confirmar'
        font_size: '20sp'
        pos_hint: {'center_x': 0.5, 'center_y': 0.25}
        size_hint: (0.7, 0.1)
        elevation: 0.5
        md_bg_color: 0.298, 0.686, 0.314, 1
        on_press: root.confirmar_registro()

```

2 - Atualiza saida

```

if response.status_code == 201:
    print("Registro salvo com sucesso!")
    # Mensagem de sucesso
    comprovante_screen = self.manager.get_screen('comprovante')
    mensagem = f"Matricula: {self.funcionario_id}\n{self.ids.data_hora.text}\n\n"
    comprovante_screen.ids.comprovante_label.text = mensagem

    # Navegar para a tela de comprovante
    self.manager.current = 'comprovante'

```

ComprovanteScreen

```

MDLabel:
    id: comprovante_label
    text: 'Comprovante'
    halign: 'center'

```

```
theme_text_color: "Primary"  
font_style: "H6"
```